

Reviewer Pack — Minimal Tropical Growth Rate vs. Circuit Depth

Entry c57147661b9c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 16:03:44 UTC

Minimal Tropical Growth Rate vs. Circuit Depth

Entry ID: c57147661b9c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 16:03:44 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For any given d -regular graph G , the minimal tropical growth rate ($\text{tgr}(G)$) of its associated Tseitin formula φ_G is linearly correlated with its circuit depth $d(\varphi_G)$, such that $\text{tgr}(G) = \Theta(d(\varphi_G))$.

Rationale (proposer's reasoning):

Tropical geometry provides a framework for studying discrete structures in a continuous manner. The tropical growth rate measures the complexity of an object as it is dequantized, which may expose underlying structure relevant to circuit complexity. A correlation between tropical growth rate and circuit depth could reveal new insights into the hardness of computational problems.

Taxonomy category: TROPICAL_GEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3151590337e73a83

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between minimal tropical growth rate ($\text{tgr}(G)$) and circuit depth ($d(\varphi_G)$) of d -regular graphs G with $n \leq 40$ vertices is ≥ 0.8 , calculated over at least 30 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (n * d) % 2 != 0:
            return None
        graph = {i: [] for i in range(n)}
        edges_added = 0
        while edges_added < n * d // 2:
            u = random.randint(0, n - 1)
            v = random.randint(0, n - 1)
            if u != v and v not in graph[u]:
                graph[u].append(v)
                graph[v].append(u)
                edges_added += 1
        return graph

    def tseitin_formula(graph):
        if graph is None:
            return []
        n = len(graph)
        literals = {i: f'x{i}' for i in range(n)}
        clauses = []
        for u in range(n):
            clause = [literals[u]]
            for v in graph[u]:
                clause.append(-literals[v])
            clauses.append(clause)
        return clauses

    def tropical_growth_rate(clauses):
        # Placeholder implementation
        return len(clauses)
```

```

def circuit_depth(clauses):
    # Placeholder implementation
    return 1

n = random.randint(5, 40)
d = random.randint(2, min(n - 1, 3))
graph = generate_d_regular_graph(n, d)
if graph is None:
    return {
        "metric_name": "tropical_growth_rate",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }
clauses = tseitin_formula(graph)
if not clauses:
    return {
        "metric_name": "tropical_growth_rate",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }
tgr = tropical_growth_rate(clauses)
depth = circuit_depth(clauses)

return {
    "metric_name": "tropical_growth_rate",
    "metric_value": tgr / depth if depth != 0 else float('inf'),
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": False,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) /
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and any(r["counterexample"] != "" for r in results):
        counterexamples = [r["counterexample"] for r in results if r["counterexample"] != ""][0]

```

```

print(f"RESULT: FALSIFIED counterexample=\"{counterexamples}\" first_failing_seed={seeds[r]
else:
print("RESULT: INCONCLUSIVE insufficient_data_or_unsupported_conjecture")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'tropical_growth_rate', 'metric_value': None, 'instances_tested': 1, 'n_max':
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d625e2ce.py", line 96, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d625e2ce.py", line 68, in run_trial
    clauses = tseitin_formula(graph)
               ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d625e2ce.py", line 44, in tseitin_formu
    clause.append(-literals[v])
                  ^^^^^^^^^^^
TypeError: bad operand type for unary -: 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's validity according to the pre-registered support condition. | next: Review and debug the test code to ensure it can run successfully and produce the required data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14483
2	preregistration	ollama_remote	glm4:latest	0	0	10668
3	novelty	ollama_remote	glm4:latest	0	0	17140
4	novelty	ollama_remote	glm4:latest	0	0	14005
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15692
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10416
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15740
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11643

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	9853

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 119641 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/c57147661b9c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c57147661b9c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c57147661b9c.tar.gz (if generated)